

CL-AWS2 — AWS avancés + serverless

Jour 1 — Du serveur à la fonction : Lambda et API Gateway

La même API StockLine qu'au TP2... sans les instances

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- **Expliquer** ce que le serverless change par rapport à l'architecture 3-tiers du TP2 (facturation, scaling, exploitation).
- **Écrire et déployer** une fonction Lambda en Python : handler, event, context, rôle d'exécution.
- **Régler** la mémoire et le timeout d'une fonction, et comprendre le cold start.
- **Lire les logs** d'une fonction dans CloudWatch et utiliser les variables d'environnement.
- **Packager** une fonction (zip, dépendances) et savoir à quoi servent les couches.
- **Exposer** une fonction en HTTP avec API Gateway (HTTP API) : routes, intégration, stage, CORS.

Fil rouge : reconstruire le `GET /produits` de StockLine — d'abord avec des données en dur, la vraie base arrive demain.

Plan de la journée

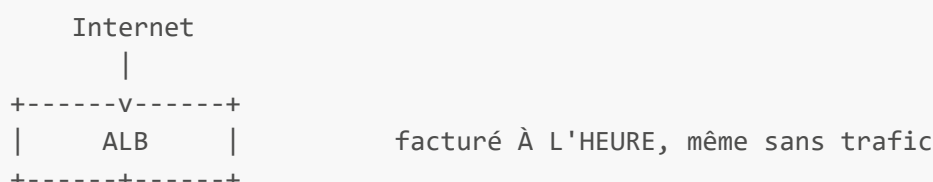
1. **Du serveur à la fonction** — ce que le TP2 coûtait à vide, la promesse serverless.
2. **Lambda** — anatomie d'une fonction Python, rôle d'exécution, mémoire/timeout, cold start, logs, variables d'environnement, packaging.
3. **Les déclencheurs** — qui invoque une fonction, et comment.
4. **API Gateway** — HTTP API vs REST API, routes, intégration Lambda, stages, CORS.

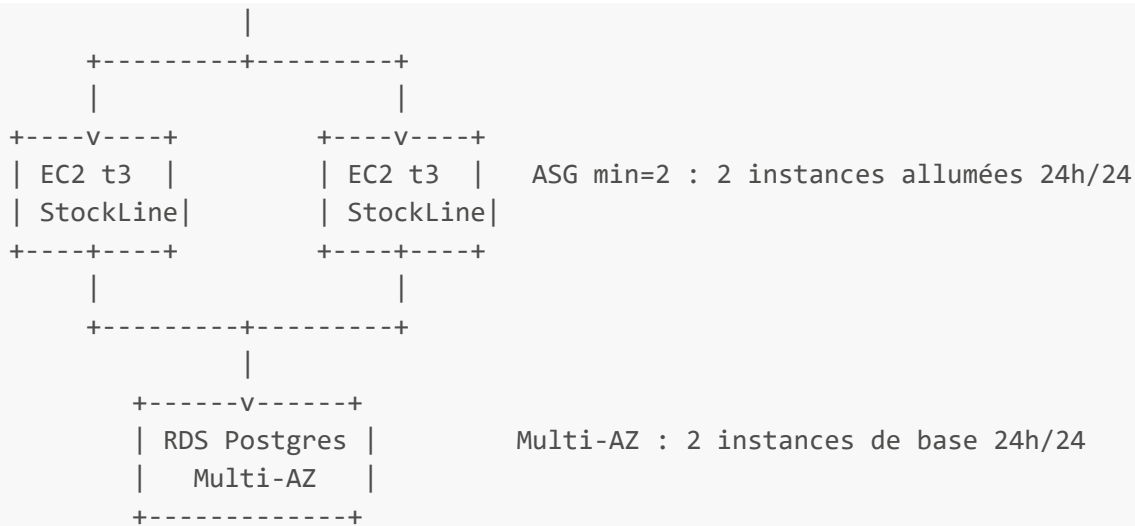
Demos : votre première Lambda (🖥️ 1-1), puis StockLine derrière API Gateway (🖥️ 1-2). Exercices : corriger un handler cassé (✏️ 1-1), dimensionner mémoire et timeout (✏️ 1-2). Quiz de fin de journée : 10 questions.

1. Du serveur à la fonction

Ce que votre ASG du TP2 vous coûtait pendant que vous dormiez

Rappel : l'architecture du TP2





Ce que cette architecture coûte... à vide

Ordres de grandeur eu-west-3, par **mois**, avec **zéro requête** (hors free-tier) :

Ressource	Facturation	Coût mensuel approx.
ALB	à l'heure + LCU	≈ 20 €
2 × EC2 t3.micro (ASG min=2)	à l'heure	≈ 17 €
RDS db.t3.micro Multi-AZ	à l'heure ×2	≈ 30 €
NAT Gateway	à l'heure + Go	≈ 33 €
Total à vide		≈ 100 €/mois

Une API interne utilisée 2 h par jour ouvré paie donc **~90 % de sa facture pour ne rien faire**. C'est le problème que le serverless attaque frontalement.

La promesse du serverless

Deux changements de modèle, pas juste une réduction de prix :

- **Payer à l'invocation** : 0 requête = 0 € de calcul. La facture suit l'usage réel, à la milliseconde d'exécution près.
- **Scaling automatique et immédiat** : 1 requête → 1 exécution ; 1 000 requêtes simultanées → ~1 000 exécutions en parallèle. Pas d'ASG à régler, pas de « min=2 pour tenir la nuit ».

Et un troisième, pour vous, admin cloud :

- **Plus d'instances à exploiter** : pas d'AMI à mettre à jour, pas d'OS à patcher, pas de disque plein à 3 h du matin. Votre travail se déplace vers les **permissions, la configuration et la supervision**.

Serverless ne veut pas dire « sans serveur » : les serveurs existent, mais ce sont ceux d'AWS, et vous ne les voyez jamais.

La boîte à outils serverless AWS — programme des 3 jours

Service	Rôle	Équivalent TP2	Jour
Lambda	exécuter du code à la demande	EC2 + votre code	J1
API Gateway	porte d'entrée HTTP	ALB	J1
DynamoDB	base de données clé-valeur	RDS PostgreSQL	J2
S3 (événements)	déclencher du code sur dépôt de fichier	cron + script	J2
SQS / SNS	files et pub/sub — découpler	(rien : tout était couplé)	J2
Step Functions	orchestrer des workflows	scripts bash enchaînés	J3
EventBridge	bus d'événements, planification	cron	J3

À la fin du J3 : le schéma complet de **StockLine serverless**, celle que vous construirez seuls au **TP3** (noté).

2. Lambda

L'unité de calcul serverless : une fonction, pas un serveur

Qu'est-ce qu'une fonction Lambda ?

Une **fonction Lambda**, c'est :

- **du code** (ici Python 3.12) avec un point d'entrée : le **handler** ;
- **une configuration** : mémoire, timeout, variables d'environnement, rôle IAM ;
- que le **service Lambda exécute pour vous** quand un événement survient.

Ce que vous ne choisissez plus : le serveur, l'OS, le scaling, la haute disponibilité (multi-AZ d'office).

Ce que vous payez : le **nombre d'invocations** + la **durée** × **mémoire** (Go-seconde).

💰 Free-tier permanent : **1 million d'invocations** et **400 000 Go-s par mois, gratuits, pour toujours**.
Toute cette semaine tiendra dedans.

Anatomie d'une fonction Python : le handler

```
import json

def handler(event, context):
    """Point d'entrée : Lambda appelle CETTE fonction à chaque invocation."""
    nom = event.get("nom", "inconnu")
    print(f"Invocation pour {nom}")          # -> logs CloudWatch
    return {
        "statusCode": 200,
```

```
"body": json.dumps({"message": f"Bonjour {nom}"}),
}
```

- **handler** : nom libre, mais déclaré dans la configuration au format `fichier.fonction` → ici `mon_fichier.handler`.
- **event** : **les données d'entrée** — un `dict` Python dont la forme dépend du déclencheur.
- **context** : **des métadonnées d'exécution** fournies par Lambda.
- La valeur de retour part au déclencheur (si invocation synchrone).

event : la forme dépend de qui appelle

Le même handler reçoit des événements très différents selon la source :

Source	Contenu typique de event
Invocation manuelle (CLI, test)	ce que vous mettez dans le JSON
API Gateway (HTTP API)	<code>routeKey</code> , <code>rawPath</code> , <code>headers</code> , <code>body</code> , <code>pathParameters</code>
S3	<code>Records[].s3.bucket.name</code> , <code>Records[].s3.object.key</code>
SQS	<code>Records[].body</code> (le message)
EventBridge planifié	<code>source</code> , <code>detail-type</code> , <code>time</code>

Premier réflexe de débogage : `print(json.dumps(event))` en début de handler, et regarder dans les logs à quoi ressemble vraiment l'événement.

context : les métadonnées de l'invocation

```
def handler(event, context):
    print(context.aws_request_id)          # id unique de CETTE invocation
    print(context.function_name)           # nom de la fonction
    print(context.memory_limit_in_mb)      # mémoire configurée
    print(context.get_remaining_time_in_millis()) # temps restant avant timeout
```

Usages concrets pour l'admin :

- **aws_request_id** : la clé de corrélation — c'est lui que vous cherchez dans CloudWatch pour suivre une requête précise.
- **get_remaining_time_in_millis()** : permet à un traitement long de s'arrêter proprement avant d'être tué par le timeout.

context n'est **pas** un dict : c'est un objet avec des attributs.

Cycle de vie : cold start et warm start

1re invocation (COLD START)	invocations suivantes (WARM)
+-----+	+-----+
1. créer le micro-environnement	
2. télécharger votre code (zip)	4. handler(event, ctx)
3. démarrer Python, imports,	... c'est tout
code HORS handler	
4. handler(event, context)	+-----+
+-----+	
~200 ms à 2 s en Python	quelques ms

L'environnement est **réutilisé** tant qu'il reste chaud (minutes à dizaines de minutes), puis détruit. Rien de ce qu'il contient n'est durable.

Cold start : ce qu'il faut en retenir

- En Python avec peu de dépendances : **200-500 ms** de cold start typique. Avec de grosses bibliothèques (pandas, numpy) : 1-3 s.
- Le cold start touche la **première** requête d'un environnement, pas les suivantes ; en trafic régulier, il devient rare.
- Conséquences pratiques :
 - **Code lourd hors du handler** (clients boto3, connexions) : payé une fois au cold start, réutilisé ensuite — c'est la bonne pratique.
 - **Jamais d'état applicatif en mémoire** : une variable globale peut survivre... ou pas. On le prouvera en démo.
- Mitigations (à connaître, pas à utiliser cette semaine) : plus de mémoire (= plus de CPU), moins de dépendances, *provisioned concurrency* (payant : des environnements pré-chauffés).

Honnêteté : pour une API interne, 300 ms sur la 1re requête est presque toujours acceptable. Pour du trading haute fréquence, non — le serverless n'est pas pour tout (tableau de décision au J3).

Mémoire et timeout : les deux réglages qui comptent

- **Mémoire** : de 128 Mo à 10 240 Mo. **Le CPU est proportionnel à la mémoire** — c'est le point contre-intuitif : on augmente parfois la mémoire pour aller plus vite, pas pour la RAM. À ~1 769 Mo, la fonction dispose d'un vCPU entier.
- **Timeout** : de 1 s à **15 minutes maximum** (limite dure). Au-delà du timeout, l'exécution est **tuée** — même au milieu d'une écriture.

Règles de dimensionnement :

1. Partir de 256 Mo / 10 s pour une API, mesurer (les logs donnent durée et mémoire max utilisée).
2. Timeout = durée observée × 3 à 5, **pas** 15 min « pour être tranquille » : un timeout large masque les problèmes et peut coûter cher en cas de boucle.
3. Un traitement qui approche 15 min n'a pas sa place dans Lambda → Step Functions (J3) ou conteneurs (J4).

 L'exercice 1-2 vous fera dimensionner 4 cas concrets.

Le rôle d'exécution : l'identité de votre code

Chaque fonction endosse un **rôle IAM** à chaque invocation. C'est **l'identité de votre code** : tout ce que la fonction peut faire sur AWS, c'est ce rôle qui le décide.

Deux parties, comme tous les rôles vus en AWS1 :

- **Politique de confiance** (*trust policy*) : QUI peut endosser le rôle → le service `lambda.amazonaws.com`.
- **Politiques de permissions** : CE QUE le code peut faire → au minimum, écrire ses logs.

```
{ "Effect": "Allow",
  "Principal": { "Service": "lambda.amazonaws.com" },
  "Action": "sts:AssumeRole" }
```

 Piège n°1 de la semaine : une fonction dont le rôle n'a pas les permissions logs **s'exécute sans laisser aucune trace**. Vous chercherez des logs qui n'existent pas.

La politique minimale : logs + rien d'autre

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EcrireSesLogsCloudWatch",
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:eu-west-3:ACCOUNT_ID:log-group:/aws/lambda/PREFIX-
produits*"
    }
  ]
}
```

- C'est l'équivalent de la politique gérée `AWSLambdaBasicExecutionRole`, mais **restreinte au log group de cette fonction** — moindre privilège dès le premier jour.
 - Chaque permission supplémentaire (DynamoDB demain, S3, SNS...) sera **ajoutée explicitement**, ressource par ressource.
 - Modèle complet : `code/08-cl-aws2/iam/policy-lambda-produits.template.json`.
-

Logs CloudWatch : automatiques (si le rôle le permet)

Tout `print()` (ou mieux : `logging`) part **automatiquement** dans CloudWatch Logs :

- Log group : `/aws/lambda/<nom-de-la-fonction>` (créé à la première invocation).
- Un *log stream* par environnement d'exécution.
- Chaque invocation ajoute 3 lignes système : `START`, `END`, `REPORT`.

```
REPORT RequestId: 8f1c... Duration: 12.34 ms Billed Duration: 13 ms
Memory Size: 256 MB Max Memory Used: 41 MB Init Duration: 312.45 ms
```

La ligne `REPORT` est votre tableau de bord : durée facturée, mémoire réellement utilisée (pour dimensionner), `Init Duration` = le cold start.

```
aws logs tail /aws/lambda/$PREFIX-produits --follow --region eu-west-3
```

Variables d'environnement

La configuration de la fonction, hors du code — exactement comme le `.env` de StockLine sur la VM du TP1 :

```
aws lambda update-function-configuration \
  --function-name $PREFIX-produits \
  --environment "Variables={TABLE_NAME=$PREFIX-stockline,NIVEAU_LOG=INFO}" \
  --region eu-west-3
```

```
import os
TABLE_NAME = os.environ.get("TABLE_NAME", "") # lu au cold start
```

- Limite : **4 Ko** au total.
- Chiffrées au repos (KMS) mais **visibles en clair dans la console** par quiconque a `lambda:GetFunctionConfiguration` → un mot de passe de base n'a rien à y faire (Secrets Manager/Parameter Store — évoqués au J3 sécurité).
- C'est la variable `TABLE_NAME` qui fera basculer notre fonction StockLine du mode « données en dur » (aujourd'hui) au mode DynamoDB (demain), **sans changer une ligne de code déployé**.

Packaging : le zip

Lambda n'installe rien : vous livrez un **paquet de déploiement** — un zip qui contient votre code **et** ses dépendances.

Cas 1 — code pur + boto3 (notre cas toute la semaine) :

```
zip -j fonction.zip lambda_produits.py
```

boto3 est **préinstallé** dans le runtime Python : rien d'autre à embarquer.

Cas 2 — dépendances externes (ex : **requests**) :

```
pip install requests --target ./paquet
cp lambda_produits.py ./paquet/
cd paquet && zip -r ../fonction.zip .
```

Limites : zip ≤ **50 Mo** (envoyé directement), ≤ **250 Mo décompressé**. Au-delà : image de conteneur (jour 4 vous donnera les clés).

Couches (layers) — en survol

Une **couche** est un zip de dépendances partagé entre fonctions, empilé sous votre code au démarrage.

```
+-----+
| votre zip (le handler) | <- change à chaque déploiement
+-----+
| couche "libs-python"   | <- pandas, requests... change rarement
+-----+
| runtime Python 3.12    | <- géré par AWS
+-----+
```

À retenir (on n'ira pas plus loin cette semaine) :

- Utile quand **plusieurs fonctions** partagent les mêmes bibliothèques, ou pour livrer un zip de code minuscule.
- Maximum **5 couches** par fonction ; la limite des 250 Mo décompressés inclut les couches.
- Ce n'est **pas** un mécanisme de sécurité ni d'isolation — juste du packaging.

Les limites Lambda à connaître par cœur

Limite	Valeur	Conséquence pratique
Durée max d'exécution	15 min	traitement long → Step Functions, conteneurs
Mémoire	128 Mo → 10 240 Mo	le CPU suit la mémoire
Zip déployé / décompressé	50 Mo / 250 Mo	grosses libs → couches ou image conteneur

Limite	Valeur	Conséquence pratique
Payload synchrone (requête/réponse)	6 Mo	pas de gros fichiers via API GW → passer par S3
Payload asynchrone	256 Ko	les événements sont des messages, pas des données
Stockage /tmp	512 Mo (jusqu'à 10 Go)	seul disque inscriptible, éphémère
Variables d'environnement	4 Ko	config oui, données non
Exécutions simultanées	1 000 par défaut (quota compte)	au-delà : throttling (erreur 429)

Ces valeurs tombent au quiz, à la certification SAA... et en entretien.



Démo 1-1

Votre première Lambda : créer, invoquer, observer

[demos/08-cl-aws2/demo-1-1-premiere-lambda.md](#)

Rôle minimal → zip → **create-function** → **invoke** → logs → mémoire éphémère prouvée

3. Les déclencheurs

Une fonction ne tourne jamais « toute seule »

Trois modes d'invocation

Mode	Qui attend la réponse ?	Exemples de sources	En cas d'erreur
Synchrone	l'appelant, en direct	API Gateway, CLI invoke , ALB	l'erreur remonte à l'appelant
Asynchrone	personne (Lambda met en file)	S3, SNS, EventBridge	2 retries automatiques, puis DLQ éventuelle
Polling	Lambda va chercher les messages	SQS, Kinesis, DynamoDB Streams	le lot revient dans la file, retraité

Pourquoi c'est important pour vous :

- Synchrone : l'utilisateur voit l'erreur → soignez les réponses.
- Asynchrone : **personne ne voit rien** → sans logs ni alarme, une fonction peut échouer en silence pendant des semaines. Supervision obligatoire.

Les sources d'événements que vous utiliserez cette semaine

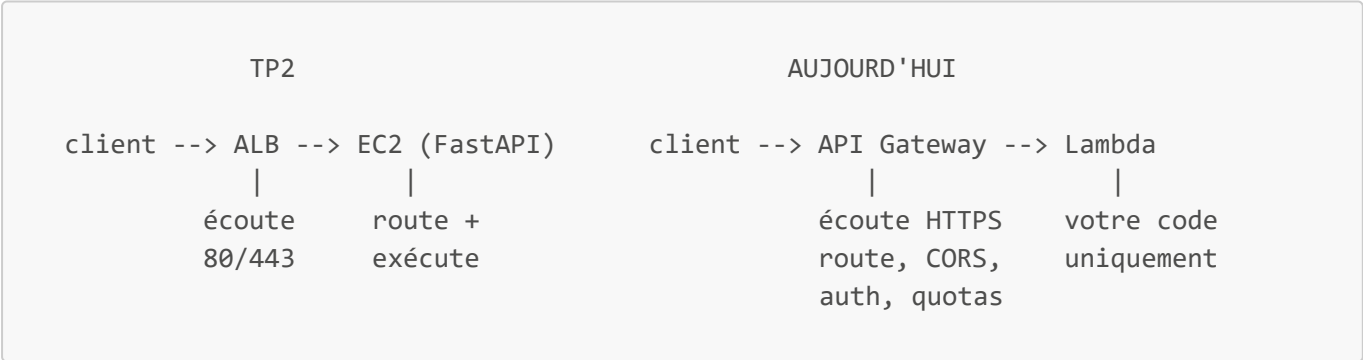
Déclencheur	Scénario StockLine	Jour
CLI / test console	mise au point, débogage	J1
API Gateway	chaque requête HTTP de l'API produits	J1
S3	un CSV d'inventaire est déposé → ingestion	J2
SQS	traitement de mouvements en file	J2
SNS	réaction à une notification	J2
Step Functions	étape d'un pipeline orchestré	J3
EventBridge (planifié)	rapport de stock chaque matin à 7 h	J3

Une même fonction peut avoir **plusieurs déclencheurs**. La question à toujours se poser : « *qui a le droit d'invoquer cette fonction ?* » — c'est une permission (resource-based policy), pas un câblage magique.

4. API Gateway

La porte d'entrée HTTP de vos fonctions

Le rôle d'API Gateway



API Gateway est un **service managé** : pas d'instance, HTTPS d'office, facturé **à la requête** (≈ 1 € le million, free-tier 1 M/mois la 1re année). Il transforme chaque requête HTTP en **event** pour la Lambda, et la réponse de la Lambda en réponse HTTP.

HTTP API vs REST API : deux produits, un service

API Gateway propose **deux types d'API** — piège classique de QCM :

	HTTP API (notre choix)	REST API
Génération	récente (v2)	historique (v1)

	HTTP API (notre choix)	REST API
Prix	~1 €/M requêtes	~3,5 €/M requêtes
Latence	plus faible	plus élevée
Intégration Lambda	oui (payload v2)	oui (payload v1)
CORS	configuration native simple	manuel (réponses OPTIONS)
Clés d'API, quotas par client, cache, validation de schéma	✗	✓
WAF, endpoints privés	✗ (partiel)	✓

Règle pratique : HTTP API par défaut ; REST API seulement si vous avez besoin d'une fonctionnalité de la colonne de droite. Cette semaine et au TP3 : **HTTP API**.

Routes : méthode + chemin → intégration

Une **route** = une clé **MÉTHODE** /**chemin** qui pointe vers une **intégration** (pour nous : une Lambda).

```

GET /sante                → Lambda produits
GET /produits             → Lambda produits
POST /produits            → Lambda produits
GET /produits/{reference} → Lambda produits   ({reference} = paramètre)
GET /stocks/{reference}   → Lambda produits
$default                 → attrape tout le reste (404 propre)

```

Deux stratégies d'architecture :

- **Une Lambda par route** : isolation maximale, mais N fonctions à gérer.
- **Une Lambda-routeur** pour l'API (notre choix) : le handler aiguille sur `event["routeKey"]` — plus proche du FastAPI que vous connaissez, plus simple à opérer au TP3.

L'intégration proxy : ce que reçoit la Lambda (payload 2.0)

`curl https://.../produits/SSD-500` devient cet event (extrait) :

```

{
  "version": "2.0",
  "routeKey": "GET /produits/{reference}",
  "rawPath": "/produits/SSD-500",
  "pathParameters": { "reference": "SSD-500" },
  "headers": { "host": "...", "user-agent": "curl/8.5.0" },
  "requestContext": { "http": { "method": "GET", "sourceIp": "203.0.113.10" } },
  "body": null,

```

```
"isBase64Encoded": false
}
```

Et le handler aiguille :

```
route = event.get("routeKey", "")
if route == "GET /produits/{reference}":
    return get_produit(event["pathParameters"]["reference"])
```

⚠ **body** est une **chaîne** (à passer dans `json.loads`), pas un dict — erreur n°1 des débutants.

Ce que la Lambda doit renvoyer

API Gateway attend un dictionnaire de cette forme :

```
{
  "statusCode": 200,
  "headers": {"Content-Type": "application/json"},
  "body": json.dumps(donnees, ensure_ascii=False), # une CHAÎNE
}
```

Les 3 erreurs qui produisent une **500 Internal Server Error** côté client :

1. Renvoyer directement `donnees` (un dict) au lieu de la structure ci-dessus.
2. Mettre un objet non sérialisable dans `body` sans `json.dumps`.
3. Lever une exception non gérée dans le handler.

Dans les trois cas, la **vraie** erreur est dans les logs CloudWatch de la fonction — jamais dans la réponse HTTP.
Réflexe : `aws logs tail`.

✏ L'exercice 1-1 vous fait corriger un handler qui cumule ces erreurs.

Stages : versions déployées de l'API

Un **stage** est un déploiement nommé de l'API, avec sa propre URL :

```
https://<api-id>.execute-api.eu-west-3.amazonaws.com/<stage>/produits
```

- Stage **\$default** : servi à la racine, sans suffixe dans l'URL — notre choix, avec **auto-deploy** (chaque modification de route est en ligne immédiatement).
- Stages nommés (**dev**, **prod**) : plusieurs versions de la même API côte à côte, avec des variables de stage (ex : **dev** → Lambda alias dev).

Pour cette semaine : `$default` + auto-deploy suffit. Retenez simplement que si votre URL renvoie `{"message": "Not Found"}`, vérifiez **le stage et la route** avant de soupçonner la Lambda.

CORS : pourquoi votre front dira « blocked by CORS policy »

Le front StockLine (HTML/JS servi depuis S3) et l'API (`execute-api....amazonaws.com`) sont sur **des origines différentes**. Le navigateur exige alors que **l'API** déclare qui a le droit de l'appeler : c'est le **CORS** (*Cross-Origin Resource Sharing*).

Sur une HTTP API, c'est une configuration native :

```
aws apigatewayv2 update-api --api-id $API_ID --region eu-west-3 \
  --cors-configuration \
    AllowOrigins="https://mon-front.s3-website.eu-west-3.amazonaws.com", \
    AllowMethods="GET,POST,OPTIONS",AllowHeaders="content-type"
```

- `AllowOrigins="*"` : acceptable en formation, **pas en production**.
- Symptôme : ça marche dans `curl` (qui ignore CORS) mais pas dans le navigateur → c'est presque toujours CORS.
- Le navigateur envoie une requête `OPTIONS` de pré-vérification : l'HTTP API y répond toute seule.



Démo 1-2

StockLine sans instance : `GET /produits` derrière API Gateway

[demos/08-cl-aws2/demo-1-2-api-gateway.md](#)

HTTP API → routes → intégration Lambda → `curl` → CORS — données en dur aujourd'hui, DynamoDB demain

TP2 vs aujourd'hui : la même API, sans les instances

TP2 (à entretenir)

```
client
|
v
[ALB]          ~20 €/mois
|
v
[EC2 x2 ASG]   ~17 €/mois
patch OS, AMI, scaling
|
v
[RDS Multi-AZ] ~30 €/mois
```

AUJOURD'HUI (à configurer)

```
client
|
v
[API Gateway]   0 € à vide
|
v
[Lambda]        0 € à vide
rôle IAM, mémoire, timeout
|
v
[demain : DynamoDB] ~0 €
```

Même contrat d'API, mêmes endpoints. Ce qui a disparu : les machines. Ce qui est apparu : **permissions, événements, configuration**.

Exercices de la journée

Exercice 1-1 — Réparer le handler (45 min) [exercices/08-cl-aws2/exercice-1-1-corriger-handler.md](#) Un handler StockLine truffé de 6 erreurs classiques (body non parsé, dict renvoyé brut, exception non gérée...) : diagnostiquez avec les logs, corrigez, redéployez.

Exercice 1-2 — Dimensionner mémoire et timeout (30 min) [exercices/08-cl-aws2/exercice-1-2-memoire-timeout.md](#) 4 cas réels (API produits, redimensionnement d'images, rapport nocturne, export 45 min) : proposez mémoire + timeout justifiés, repérez le cas qui ne rentre pas dans Lambda.

Corrigés complets dans [solutions/08-cl-aws2/](#) — après avoir cherché.

Récap — les points clés du jour

- Le TP2 coûtait **≈ 100 €/mois à vide** ; le serverless facture à **l'invocation** et scale tout seul.
 - Une fonction Lambda = **handler(event, context)** + configuration + **rôle d'exécution** (l'identité du code — moindre privilège dès le premier jour).
 - **Cold start** : init payée à la première invocation d'un environnement ; clients boto3 hors du handler ; **jamais d'état en mémoire**.
 - **Mémoire** = **CPU** ; timeout max **15 min** ; ligne **REPORT** des logs pour dimensionner.
 - Packaging = **zip** (code + dépendances, boto3 fourni) ; couches pour partager des libs.
 - **API Gateway HTTP API** : routes **MÉTHODE /chemin** → intégration Lambda proxy ; la Lambda renvoie **statusCode** + **body** (chaîne JSON) ; **CORS** natif.
 - Erreur 500 côté client → la vérité est dans **CloudWatch Logs**, pas dans la réponse.
-

Quiz de fin de journée

10 questions — 15 minutes

Réponses commentées demain matin

Quiz — questions 1 et 2

Question 1 — Dans l'architecture du TP2, quelles ressources facturaient à l'heure même sans aucune requête ? Citez-en trois, et expliquez ce que le modèle serverless change pour chacune.

Question 2 — Votre fonction est configurée avec `--handler lambda_produits.handler`. Que désignent exactement les deux parties de cette valeur, et que se passe-t-il si le fichier zippé s'appelle `main.py` ?

Quiz — questions 3 et 4

Question 3 — Quelle est la différence entre **event** et **context** dans un handler ? Donnez un exemple concret d'information trouvée dans chacun.

Question 4 — Une fonction Python met 1,8 s à répondre à sa première invocation, puis 40 ms aux suivantes. Expliquez le phénomène, et citez deux bonnes pratiques de code qui en tirent parti ou en limitent l'effet.

Quiz — questions 5 et 6

Question 5 — Pourquoi augmenter la mémoire d'une fonction Lambda peut-il la rendre **plus rapide**, alors qu'elle n'utilise que 60 Mo de RAM ? Quelle ligne des logs vous donne la mémoire réellement utilisée ?

Question 6 — Vous invoquez une fonction fraîchement créée : elle répond correctement, mais le log group `/aws/lambda/ma-fonction` n'existe pas dans CloudWatch. Quelle est la cause la plus probable, et comment la corriger ?

Quiz — questions 7 et 8

Question 7 — Citez trois limites chiffrées de Lambda (durée, payload synchrone, taille du zip) et, pour chacune, la solution quand on la dépasse.

Question 8 — Vous devez exposer une API interne simple, au meilleur coût, avec CORS. HTTP API ou REST API ? Justifiez, et citez deux fonctionnalités qui imposeraient l'autre choix.

Quiz — questions 9 et 10

Question 9 — Un `curl` sur votre route renvoie `{"message": "Internal Server Error"}`. Le code de votre handler semble correct. Décrivez votre démarche de diagnostic, dans l'ordre, avec les commandes.

Question 10 — Votre API répond parfaitement dans `curl`, mais le front StockLine dans le navigateur affiche une erreur `blocked by CORS policy`. Expliquez pourquoi `curl` fonctionne quand même, et donnez la correction côté HTTP API.

À demain

Jour 2 — Données et découplage

Aujourd'hui, notre `GET /produits` renvoie des données **en dur** — et le POST renvoie un 501 assumé.

Demain : **DynamoDB** (la base sans serveur, et pourquoi on fuit le `scan`), les **événements S3** (un CSV déposé qui se charge tout seul), **SQS et SNS** pour découpler — et en fin de journée, votre premier **pipeline complet** : CSV → S3 → Lambda → DynamoDB → notification. 📦